CS101
StringMethods

4/21

Announcements

1. Thursday 4/23:
    a. Continue Lab14-fileScanner
    b. Release HW5-TopSongs
2. Friday 4/24:
    a. Lab11-Arrays
    b. HW4-PickachuBattle

Agenda

1. Oral Quiz practice2
2. String Comparison
3. Lab15-StringMethods
4. 1:1s with Mr.M
5. Quiz11-Practice
6. HW4 Qs

Oral Quiz Practice 2

STRING COMPARISON IN JAVA

Always use .equals() to compare Strings in Java. The == operator compares **memory addresses** (references), not the actual text content.

WHY == FAILS

```
String a = "Hello";
String b = new String("Hello");

System.out.println(a == b);      // false — different objects in memory
System.out.println(a.equals(b)); // true  — same characters inside
```

→ == asks: "Are these the **same object** in memory?"

→ .equals() asks: "Do these contain the **same text**?"

TRICKY! == SOMETIMES WORKS

```
String x = "Hello";
String y = "Hello";

System.out.println(x == y); // true!
```

Java **interns** identical string literals, reusing the same object. This makes == appear to work — but it's unreliable.

WHEN == BREAKS

```
// User input, concatenation, or new String()
// all create NEW objects in memory

Scanner sc = new Scanner(System.in);
String input = sc.nextLine();
// Even if user types "Hello":
"Hello" == input  // false!
```

Strings from user input, new String(), or concatenation are different objects — == will return false.

.EQUALS() — CORRECT WAY

```
String a = "Hello";
String b = "hello";

a.equals(b)           // false (case sensitive)
a.equalsIgnoreCase(b) // true
```

.equals() is case-sensitive. Use .equalsIgnoreCase() when casing doesn't matter.

WHY THE DIFFERENCE?

```
// Primitives store VALUES directly
int a = 5;
int b = 5;
a == b  // true — compares values

// Strings are OBJECTS (references)
String x = "Hi";
String y = "Hi";
x == y  // compares references, NOT text
```

Lab15 - StringMethods

Lab14 - FileScanner